

Zero Label Cropper (Cropper Hub) - Website & Technology Overview

Architecture, frameworks, libraries, and deployment stack

1. Product summary

Zero Label Cropper (Cropper Hub) is a web application for e-commerce sellers working with Meesho and Flipkart shipping-label PDFs. It provides automated cropping and sorting, optional OCR exports (CSV/Excel), premium labeling features (suspicious-buyer detection, pincode splits, loyalty markers, custom print fields), coin-based billing via an in-app wallet, user dashboards, admin tools, and secure download of processed PDFs or ZIP archives.

2. Frontend technologies

Core framework and tooling

- JavaScript runtime: ES modules (type: module in package.json).
- UI library: React 18.3.x (react, react-dom).
- Build tool and dev server: Vite 5.4.x with @vitejs/plugin-react (Fast Refresh, optimized bundles).
- Routing: react-router-dom 7.x (BrowserRouter, lazy-loaded route components, nested layouts).
- Client-side PDF utilities: pdf-lib 1.17.x (when PDF manipulation is needed in the browser).
- Firebase SDK: firebase 12.x (included in dependencies for optional integrations).
- Code style: Prettier 3.x for formatting src/**/*.{js,jsx}.
- HTTP: native fetch API for REST calls to the backend (no axios in package.json).

Application structure

- State: React Context for authentication (AuthContext) and wallet (WalletProvider).
- Session: Bearer token stored in localStorage (cropperhub_session); sent as Authorization header.
- Config: import.meta.env.VITE_API_URL for production API base URL; Vite dev proxy forwards /api to localhost FastAPI (VITE_DEV_API_PROXY override).
- Optional env: VITE_ARTIFACT_DOWNLOAD_TIMEOUT_MS for large artifact downloads.
- Major views: lazy-loaded pages (HomePage, WorkspacePage, LoginPage, SignupPage, MyDashboardPage, WalletPage, AdminPage, HistoryPage, etc.).
- Large feature components: WorkspaceView (upload + async tasks), MeeshoNeonView (Meesho workspace UI), Header, ProtectedRoute, AdminRoute, AppErrorBoundary.
- API module: src/api/taskApi.js (fetchWithRetry, fetchTaskStatus, downloadTaskArtifact with as_json=1 for presigned R2 URLs).

Hosting (frontend)

- Static SPA built with npm run build (vite build); deployed to Vercel.
- vercel.json: SPA fallback rewrites so client-side routes (e.g. /login, /meesho) resolve correctly.

3. Backend technologies

Web framework and server

- Language: Python 3.11 (Dockerfile base image python:3.11-slim).
- ASGI framework: FastAPI 0.115+ with Pydantic models for request/response validation.
- HTTP server: Uvicorn 0.32+ with standard extras (WebSockets support where applicable).
- Multipart uploads: python-multipart for file upload endpoints.
- CORS: Starlette CORSMiddleware with configurable allow_origins (environment-driven).

Python dependencies (requirements.txt)

- fastapi: REST API, dependency injection, OpenAPI docs.
- uvicorn[standard]: production ASGI server.
- python-multipart: multipart form parsing for PDF uploads.
- pymupdf (PyMuPDF): PDF open, render, crop, merge, page extraction (fitz).
- pdfplumber: supplementary PDF text/layout parsing where needed.
- pytesseract: OCR bridge to Tesseract engine.
- Pillow (PIL): image handling for stamps/markers and OCR prep.
- openpyxl: Excel workbook generation for OCR/exports.
- redis (pinned <5): Redis queue client (RESP2 compatibility with some hosted proxies).

Zero Label Cropper (Cropper Hub) - Website & Technology Overview

Architecture, frameworks, libraries, and deployment stack

- psycopg[binary]: PostgreSQL driver when DB_BACKEND=postgres / Neon etc.
- boto3: AWS SDK-compatible client for S3 API (Cloudflare R2, presigned URLs).

System dependencies (Docker image)

- tesseract-ocr, tesseract-ocr-eng: OCR engine for label OCR pipelines.
- libgl1, libgl2.0-0: native libs commonly required by Pillow/OpenCV-style stacks.
- Container exposes port 8000; Railway uses PORT env with start.sh for uvicorn bind.

Backend modules (conceptual)

- server.py: FastAPI app, routes, middleware, startup hooks.
- auth_store.py: users, sessions, wallet; SQLite or optional Postgres.
- history_store.py: crop_jobs, metrics, history APIs.
- task_queue.py: enqueue, worker logic, Redis/SQLite backends, S3 result upload.
- worker.py: standalone consumer for VPS Docker Compose.
- hybrid/storage.py, hybrid/queue.py: auxiliary abstractions for object storage and Redis.
- Domain logic: meesho_service.py, flipkart_service.py, partner_crop.py, pdf_sort_delivery.py, pdf_sort_sku.py, label_ocr_service.py, return_analysis_service.py.

Zero Label Cropper (Cropper Hub) - Website & Technology Overview

Architecture, frameworks, libraries, and deployment stack

4. Data stores and protocols

- SQLite: default embedded DB file (LABELHUB_DB_PATH; e.g. /app/data/labelhub.db on Railway with volume). Stores users, sessions, wallet ledger, crop_jobs, processing_tasks shadow rows.
- PostgreSQL: optional via DATABASE_URL and DB_BACKEND=postgres for auth (Neon, RDS-style).
- Redis: LIST/STRING/ZSET patterns for queue and task metadata when QUEUE_BACKEND=redis; Railway internal or public URL.
- Cloudflare R2: S3-compatible object storage (STORAGE_BACKEND=s3, boto3, S3_* env vars); presigned GET for downloads; bucket CORS for browser origins.
- HTTP APIs: REST + JSON; Bearer token authentication; optional Idempotency-Key headers on some endpoints.
- File formats: PDF (primary), ZIP (multi-file outputs), CSV/XLSX for OCR exports.

5. Deployment and infrastructure

- Frontend: Vercel (global CDN, HTTPS, automatic builds from Git).
- API: Railway (Docker deploy from repo Dockerfile; internal networking; public HTTPS URL).
- Redis: Railway Redis plugin or equivalent managed Redis.
- Workers: Linux VPS (e.g. Hetzner) running Docker Compose (deploy/hybrid/docker-compose.worker.yml): multiple worker replicas, optional custom DNS resolvers.
- Domain/DNS: user domain on Vercel; API subdomain or Railway-provided hostname.
- Secrets: environment variables only (never committed); rotation recommended for Redis and R2 keys.
- Observability: /api/health and application logging; optional ops scripts under scripts/ops/.

6. End-to-end processing flow

1. Browser uploads PDFs to FastAPI (multipart).
2. API authenticates session, creates crop_job row, may upload inputs to R2, enqueues Redis task, writes shadow SQLite row for UI joins.
3. VPS worker pulls task, downloads inputs from R2 if needed, runs PyMuPDF/Tesseract pipelines, uploads output to R2, updates Redis task to success.
4. Browser polls GET /api/tasks/{id}; API merges Redis state into SQLite for history.
5. GET /api/tasks/{id}/download?as_json=1 returns JSON with presigned URL; browser fetch() downloads from R2 without embedding secrets.

7. Development workflow

- Frontend: npm run dev (Vite) with proxy /api -> local uvicorn.
- Backend: cd backend && python -m uvicorn server:app --reload --port 8000 (per server.py docstring).
- Production parity: Docker build matches Railway; worker compose file for hybrid queue testing.
- Formatting: npm run format (Prettier).

8. Document revision

This PDF is generated from scripts/generate_website_overview_pdf.py using the fpdf2 library. Regenerate after dependency or architecture changes: python scripts/generate_website_overview_pdf.py